GANGA888.IN · AWS AP-SOUTH-1 · PRODUCTION DESIGN

ShopVerse: a 15-service e-commerce platform on AWS

ShopVerse is an online store split into 15 independently deployed microservices running on Amazon EKS. Customers reach it at `ganga888.in` through CloudFront and an Application Load Balancer. Orders run as a saga across order, inventory and payment over Kafka. Every change to code or infrastructure reaches AWS through Git: Jenkins and GitHub Actions build and plan, and ArgoCD and Terraform apply.

15
microservices

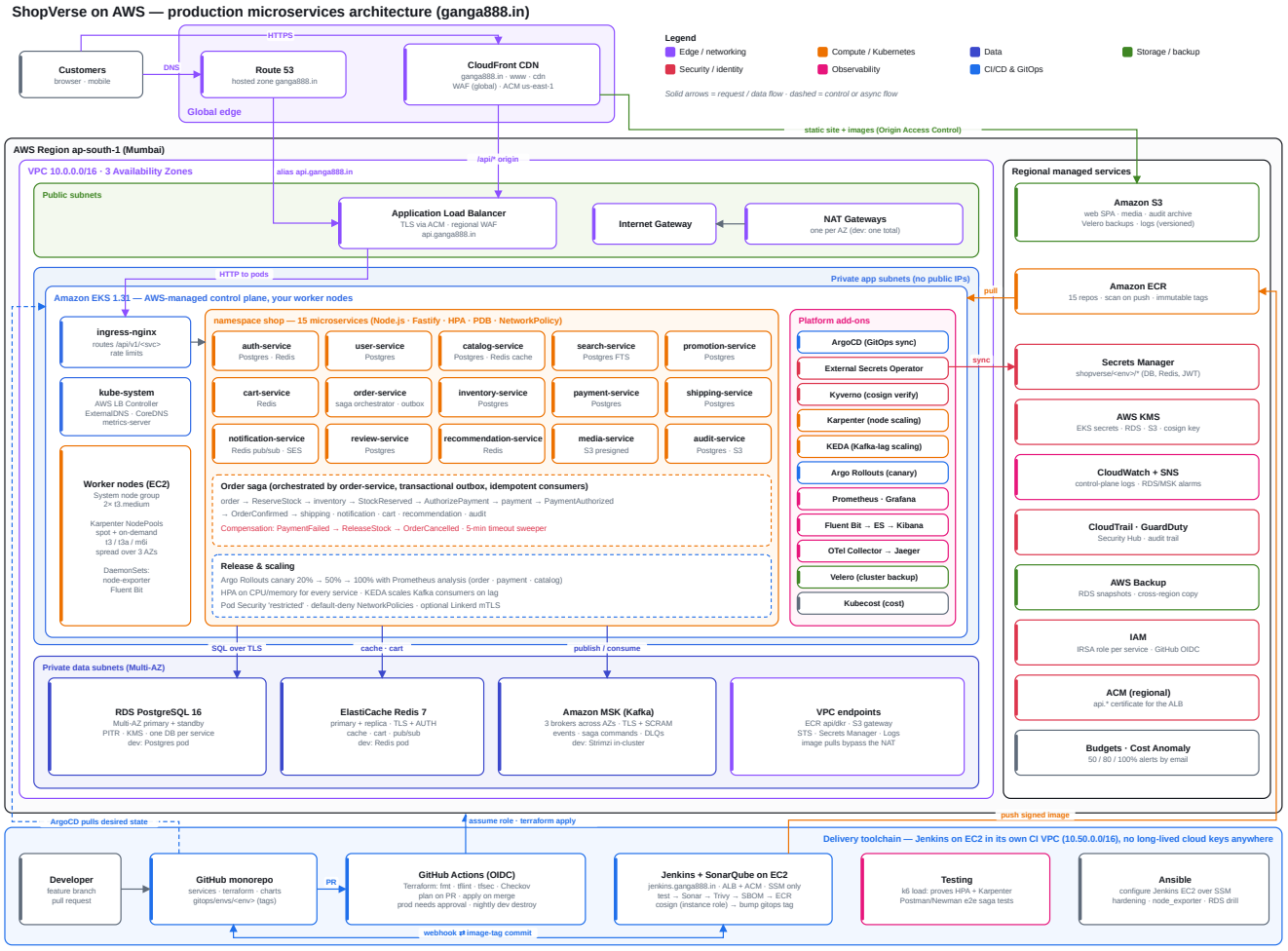
3
environments: dev, staging, prod

3
availability zones in prod

9
Kafka topics + dead-letter topics

0
AWS keys stored in Git

Architecture diagram



Traffic enters at the top and moves down through the VPC layers: edge, public subnets, the EKS cluster, then the data stores. The right column holds regional services that live outside the VPC. The bottom strip is the delivery toolchain, which only reaches AWS through OIDC roles, ECR pushes and ArgoCD pulling from Git.

The downloadable files sit in the project folder under `aws-microservices/docs/diagrams/`. Each diagram comes as an SVG, which scales to any size and can be imported into draw.io or Lucidchart, and as a 2× PNG for slides and documents.

Architecture

`docs/diagrams/architecture.svg · .png`

CI/CD, GitOps and Terraform

`docs/diagrams/cicd-gitops.svg · .png`

Order saga sequence

`docs/diagrams/saga.svg · .png`

Whole repo

`shopverse-aws-microservices.zip`

How a request flows

This is the path of a customer placing an order on `ganga888.in`. Each hop maps to a box in the diagram.

1. **Route 53** resolves `ganga888.in` to CloudFront and `api.ganga888.in` to the ALB (alias records that ExternalDNS keeps up to date).
2. **CloudFront** serves the single-page web app from the private S3 web bucket. Its global **WAF** drops SQL injection, XSS, known-bad IPs and floods before they reach AWS in Mumbai. Calls to `/api/*` are forwarded to the ALB.
3. The **ALB** terminates TLS with an **ACM** certificate, applies the regional WAF and forwards to the **ingress-nginx** pods. It was created by the AWS Load Balancer Controller from a Kubernetes Ingress, so nobody clicks it together in the console.
4. **ingress-nginx** routes `/api/v1/orders` to `order-service`, applies per-client rate limits and adds request IDs.
5. **order-service** verifies the customer's JWT against the auth-service public key (JWKS). In a single Postgres transaction it saves the order as `PENDING` and writes the saga commands to an outbox table.
6. The outbox relay publishes `ReserveStock` to **MSK**. inventory, then payment, answer through Kafka. The order becomes `CONFIRMED`, or `CANCELLED` with compensation.
7. Every hop emits a **trace** (OpenTelemetry to Jaeger), **metrics** (Prometheus) and a **JSON log** line with the same `trace_id` (Fluent Bit to Elasticsearch), so one slow checkout can be followed across all six services.

The 15 microservices

All services are Node.js 22 with TypeScript and Fastify. They share one internal library for logging, metrics, tracing, Kafka and health checks. Each service owns its data: one Postgres database per service on a shared RDS instance, separate Redis key prefixes, and no service reads another's tables.

SERVICE	WHAT IT DOES	DATA	TALKS OVER KAFKA
auth-service	Sign-up, login, RS256 JWTs, refresh tokens, JWKS endpoint other services trust	Postgres `auth`, Redis	publishes UserRegistered

SERVICE	WHAT IT DOES	DATA	TALKS OVER KAFKA
user-service	Profiles and saved addresses	Postgres `users`	consumes users.events
catalog-service	Products, categories and prices, with Redis cache-aside	Postgres `catalog`, Redis	publishes catalog.events
search-service	Full-text product search, kept in sync from catalog events	Postgres `search` (tsvector)	consumes catalog.events
inventory-service	Stock levels; reserve, release and commit with row locks	Postgres `inventory`	saga participant
cart-service	Shopping cart with expiry	Redis	clears cart on OrderConfirmed
order-service	Checkout and the saga orchestrator, with outbox and a timeout sweeper	Postgres `orders`	drives the saga
payment-service	Payment authorisation through a gateway (mocked), with idempotency keys and refunds	Postgres `payments`	saga participant
shipping-service	Creates and tracks shipments for confirmed orders	Postgres `shipping`	consumes orders.events
notification-service	Email through SES and live order updates (Redis pub/sub to a browser stream)	Redis	consumes orders, shipping, users
review-service	Product ratings and reviews	Postgres `reviews`	publishes reviews.events
promotion-service	Coupons and discount rules applied at checkout	Postgres `promotions`	none
recommendation-service	"Frequently bought together" from order history	Redis sorted sets	consumes orders.events
media-service	Presigned S3 upload and download URLs for product images, served via cdn.ganga888.in	S3 media bucket	none
audit-service	Immutable record of every business event, archived to S3 as JSONL	Postgres `audit`, S3	consumes all events

AWS services and why each one is here

Each row covers what the service does, why ShopVerse needs it, and how it is set up. The dev column shows the cheaper substitute used while learning.

SERVICE	WHY SHOPVERSE USES IT	HOW IT IS CONFIGURED	DEV SUBSTITUTE
VPC NETWORK	A private network so databases and pods have no public IPs. Only the load balancer is internet-facing.	10.0.0.0/16 across 3 AZs: public, private-app and private-data subnets. Flow logs on. Subnets tagged for the ALB and Karpenter.	Same layout, 2 AZs allowed
NAT Gateway NETWORK	Lets private pods reach the internet (package mirrors, payment API) without being reachable from it.	One per AZ in prod so one AZ failing doesn't cut egress for the others. VPC endpoints carry ECR, S3 and STS traffic so image pulls don't pay NAT data charges.	One NAT total
EKS COMPUTE	Managed Kubernetes. AWS runs the control plane across 3 AZs; we run 15 services plus the platform add-ons.	Kubernetes 1.31, KMS-encrypted secrets, control-plane logs to CloudWatch, access entries instead of the aws-auth ConfigMap, IRSA for pod permissions.	Destroyed nightly ($0.10/hr)
EC2 COMPUTE	The worker nodes pods actually run on.	A small managed node group (2× t3.medium) for system add-ons. Everything else goes on Karpenter-launched nodes that mix spot and on-demand and are right-sized to pending pods.	Spot only
ALB EDGE	Terminates HTTPS for `api.*`, runs the regional WAF and spreads traffic across AZs.	Created by the AWS Load Balancer Controller from an Ingress. Targets ingress-nginx pods directly (IP mode). HTTP redirects to HTTPS.	Same
RDS PostgreSQL DATA	The system of record for 10 services. AWS handles backups, patching and failover.	Postgres 16, Multi-AZ, point-in-time recovery, KMS, forced SSL, Performance Insights. One database and one user per service. Terraform puts the credentials in Secrets Manager and an in-cluster db-init job creates the roles.	Postgres pod
ElastiCache Redis DATA	Catalog cache, carts, refresh tokens, recommendations, and pub/sub fan-out for live notifications.	Redis 7 replication group, Multi-AZ automatic failover, in-transit TLS with an AUTH token.	Redis pod
MSK (Kafka) DATA	The durable event log. Services publish facts and the saga sends commands, so a slow consumer never blocks checkout.	3 brokers in 3 AZs, TLS with SASL/SCRAM-SHA-512 credentials from Secrets Manager, topic auto-create off, replication factor 3, dead-letter topic per topic.	Strimzi in-cluster
S3 STORAGE	Web app files, product images, audit archive, Velero backups and logs.	All private with versioning and KMS. CloudFront reads through Origin Access Control. Lifecycle rules move old objects to cheaper tiers.	Same

SERVICE	WHY SHOPVERSE USES IT	HOW IT IS CONFIGURED	DEV SUBSTITUTE
ECR COMPUTE	Private registry Jenkins pushes images to and EKS pulls from.	One repo per service, immutable tags, scan on push, keeps the last 30 images.	Shared
Route 53 EDGE	DNS for ganga888.in and its subdomains.	One hosted zone. Point your registrar's NS records at it once. ExternalDNS manages the records; ACM validates certificates through it.	Shared zone
CloudFront EDGE	Serves the web app and images from edge locations close to customers, and shields the origin.	Two distributions: the site (S3 plus `/api/*` to the ALB) and the media CDN. HTTP/2 and 3, TLS 1.2+.	Same
ACM SECURITY	Free, auto-renewing TLS certificates.	One certificate in us-east-1 for CloudFront (a CloudFront requirement) and one in ap-south-1 for the ALB, both DNS-validated.	Same
WAF SECURITY	Blocks SQL injection, XSS, bad bots and request floods before they reach pods.	AWS managed rule groups (Common, KnownBadInputs, SQLi, IP reputation) plus a rate-limit rule, on both CloudFront and the ALB.	Count mode
Secrets Manager SECURITY	The one place passwords, the JWT signing key and the payment API key live. Nothing sensitive goes into Git.	Paths `shopverse/<env>/…`. External Secrets Operator syncs them into Kubernetes Secrets. RDS master password is AWS-managed and rotated.	Same
KMS ADDED	Encryption keys you control, and the key that signs container images.	Separate keys for EKS secrets, RDS, S3 and cosign signing, with rotation on.	Same
IAM + OIDC ADDED	Least privilege for every pod and pipeline, with no long-lived keys.	IRSA role per service that needs AWS (media, audit, notification). GitHub Actions assumes roles through OIDC.	Same
CloudWatch + SNS ADDED	Metrics and alarms for the managed services Prometheus can't see inside.	Alarms for RDS CPU, storage and connections, Redis memory, MSK disk. Alerts go to SNS email.	Fewer alarms
CloudTrail, GuardDuty, Security Hub ADDED	An audit trail of every API call, threat detection, and a security posture score.	Multi-region trail to an encrypted S3 bucket. GuardDuty with EKS protection. Security Hub foundational best practices.	Account-wide
AWS Backup ADDED	Backups outside the database's own retention, copied to a second region.	Daily RDS backup plan with a cross-region copy for prod.	Off

SERVICE	WHY SHOPVERSE USES IT	HOW IT IS CONFIGURED	DEV SUBSTITUTE
Budgets + Cost Anomaly ADDED	An email before a forgotten cluster becomes a big bill.	Alerts at 50, 80 and 100% of the monthly budget, plus anomaly detection.	Same

Tools and why each one is here

TOOL	WHY SHOPVERSE USES IT	WHERE IT RUNS AND HOW
Terraform INFRA	All AWS infrastructure as code. The same modules build dev, staging and prod, and `terraform destroy` is the cost safety valve.	`terraform/modules/*` wired by `terraform/envs/<env>`. State in S3 with a DynamoDB lock. Run only by GitHub Actions.
Docker BUILD	Packages each service with its dependencies so the image tested in CI is the one that runs in prod.	Multi-stage Dockerfiles, small Alpine runtime, non-root user, read-only root filesystem.
Kubernetes RUNTIME	Schedules pods, restarts crashed ones, scales them and load-balances between them.	Namespace `shop` for services. Every pod has probes, requests and limits, a PDB and zone spread.
Helm PACKAGING	One generic chart for all 15 services instead of hundreds of YAML files. Upstream charts install the platform add-ons.	`charts/microservice`, with per-service, per-environment values in `gitops/envs`.
ArgoCD GITOPS	Makes the cluster match Git. Deploy means a commit and rollback means `git revert`. It also flags drift when someone edits the cluster by hand.	In-cluster, app-of-apps per environment, an ApplicationSet generating one app per service, auto-sync with self-heal.
GitHub Actions CI/CD	Terraform pipeline: plan on pull request, apply on merge. Also handles promotion PRs, web deploys and the nightly dev shutdown.	OIDC into AWS. Prod apply waits for a required reviewer in the `prod` environment.

TOOL	WHY SHOPVERSE USES IT	WHERE IT RUNS AND HOW
Jenkins CI	The app build pipeline: test, scan, build, push to ECR, then bump the tag in GitOps.	Runs in Docker on an EC2 host (`terraform/ci`) in its own CI VPC, behind an ALB at `https://jenkins.ganga888.in` with an ACM certificate and an admin-IP allowlist. No SSH (SSM only) and no access keys: the instance role pushes to ECR and signs with KMS. Configured as code (JCasC). Only changed services build, in parallel.
SonarQube QUALITY	Static analysis that catches bugs, code smells and missing test coverage before merge.	Runs on the same EC2 host at `https://sonarqube.ganga888.in` . The quality gate fails the build.
Trivy SECURITY	Scans each image for known CVEs before it is pushed.	Fails on fixable HIGH or CRITICAL findings. Also scans Terraform and Kubernetes config.
tfsec + Checkov SECURITY	Catch insecure infrastructure, such as a public bucket or an unencrypted database, in the Terraform PR.	GitHub Actions, before plan.
cosign + syft ADDED	Sign each image and record its SBOM, so the cluster only runs images Jenkins built.	Signs with the KMS key. Kyverno verifies the signature at admission.
Ansible CONFIG	Configures machines Terraform created but doesn't manage inside: the Jenkins host and ops runbooks.	Connects to the Jenkins EC2 host over SSM (no SSH). Roles for hardening, Docker, the Jenkins stack, node_exporter and the CloudWatch agent; secrets are read on the host from Secrets Manager. Playbook for the RDS restore drill.
Prometheus + Node Exporter METRICS	Scrape and store request rate, errors, latency, saga outcomes, Kafka lag and node CPU and memory.	kube-prometheus-stack. Node Exporter runs as a DaemonSet on every node. Alertmanager sends burn-rate SLO alerts.
Grafana DASHBOARDS	Dashboards for service health, saga funnel, Kafka lag, capacity, and RDS through CloudWatch.	`grafana.<env>.ganga888.in` , dashboards stored as code.
EFK LOGS	Fluent Bit ships every pod's JSON logs to Elasticsearch. Kibana searches them by `trace_id` , order ID or service.	ECK operator, 3 nodes in prod with 30-day retention (7 days in dev).

TOOL	WHY SHOPVERSE USES IT	WHERE IT RUNS AND HOW
Jaeger + OpenTelemetry TRACES	Shows which of the six services in a checkout was slow.	Services export OTLP to an OpenTelemetry Collector, which forwards to Jaeger and derives span metrics.
External Secrets Operator SECRETS	Turns Secrets Manager entries into Kubernetes Secrets, so Git holds only references.	ClusterSecretStore `aws-secrets-manager` using IRSA, with a 1-hour refresh.
HPA, Karpenter, KEDA AUTOSCALING	Scaling at every layer. HPA adds pods on CPU, KEDA adds consumer pods when Kafka lag grows, and Karpenter adds nodes when pods can't be placed.	HPA on all services. KEDA on inventory, payment, shipping, notification and audit. Karpenter consolidates empty nodes.
Argo Rollouts RELEASE	Canary releases that roll back on their own if error rate or latency gets worse.	order, payment and catalog in staging and prod: 20%, then 50%, then 100%, with Prometheus analysis between steps.
Kyverno ADDED	Admission policies: signed images only, no `:latest`, limits and probes required, ECR only.	Enforce mode in staging and prod, audit mode in dev.
Velero BACKUP	Backs up cluster objects and volumes, so a lost cluster can be rebuilt.	Daily full backup kept 30 days, hourly `shop` namespace backup kept 48 hours, stored in S3.
Kubecost COST	Cost per service and namespace, and which requests are oversized.	Reads from Prometheus. Pairs with tagging and AWS Budgets.
k6 TESTING	Load tests that prove HPA and Karpenter scale and p95 stays under 500 ms.	Smoke, load, stress, spike and soak scripts with pass/fail thresholds.
Postman / Newman TESTING	End-to-end API tests, including the full checkout saga and the payment-failure compensation path.	Collection plus dev, staging and prod environment files, run by Newman after deploys.
Linkerd (optional) MESH	Mutual TLS between all pods and per-route golden metrics, without code changes.	Included but disabled. Turn it on once the basics are stable.

What I added beyond your list

These fill gaps that would otherwise surface in production.

AWS Load Balancer Controller + ExternalDNS

Without them, the ALB and DNS records are built by hand and drift from the cluster. With them, an Ingress is the only thing you write.

OpenTelemetry Collector

One pipe for traces. Services stay vendor-neutral, and span metrics feed Prometheus.

Transactional outbox + idempotent consumers

A saga without these loses or duplicates events whenever a pod restarts mid-write.

Dead-letter topics

A poison message goes to `<topic>.dlq` after retries instead of blocking the partition.

VPC endpoints

ECR, S3, STS and Secrets Manager traffic stays private and skips NAT data charges.

Kyverno + cosign + SBOM

Closes the supply chain: only images that Jenkins scanned and signed can start.

KMS, CloudTrail, GuardDuty, Security Hub

Encryption you control, plus the audit trail and threat detection a real account needs.

AWS Backup with cross-region copy

Database backups survive the loss of the whole region or the RDS instance.

Strimzi and in-cluster data for dev

Dev runs Kafka, Postgres and Redis as pods (`use_managed_data = false`), which removes most of the managed-data cost while learning.

Nightly dev destroy and morning recreate

Scheduled GitHub Actions jobs. ArgoCD restores every app from Git when the cluster comes back.

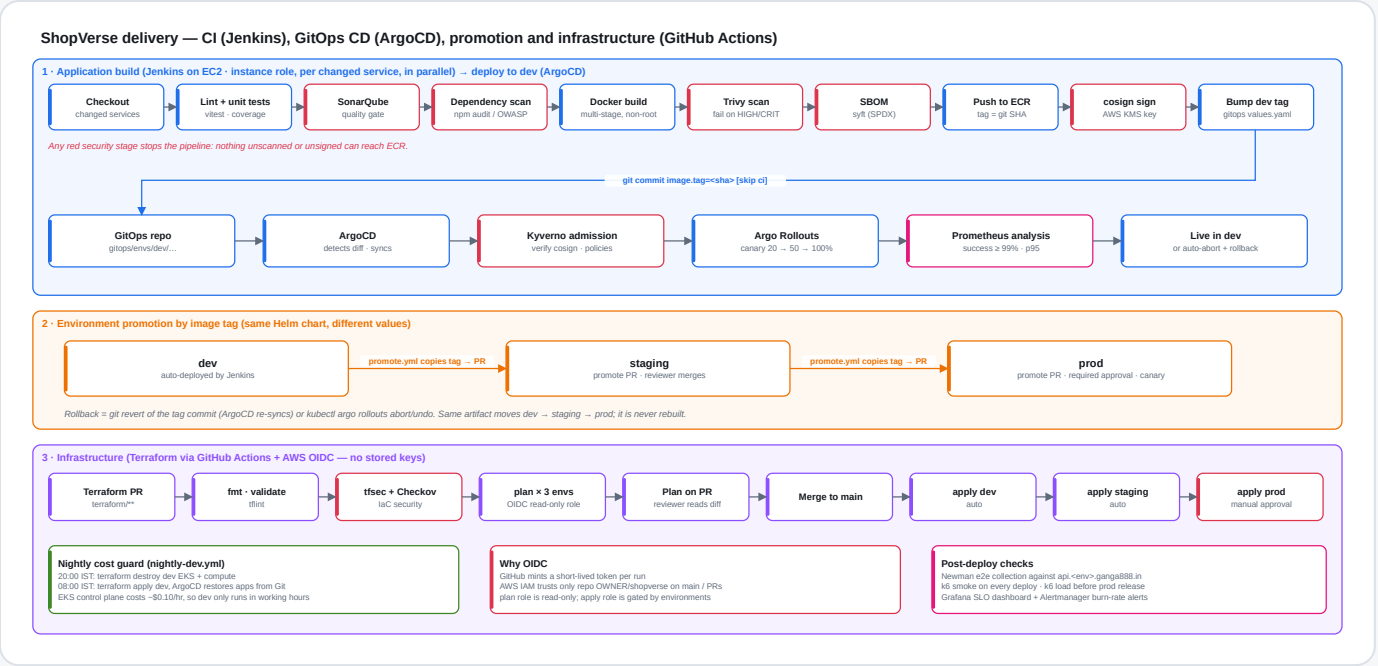
SLO burn-rate alerts

Page when the 99.9% availability budget is burning fast, rather than on every CPU spike.

Runbooks

DR restore, incident response, rollback, secret rotation, scaling, promotion, and onboarding a new service.

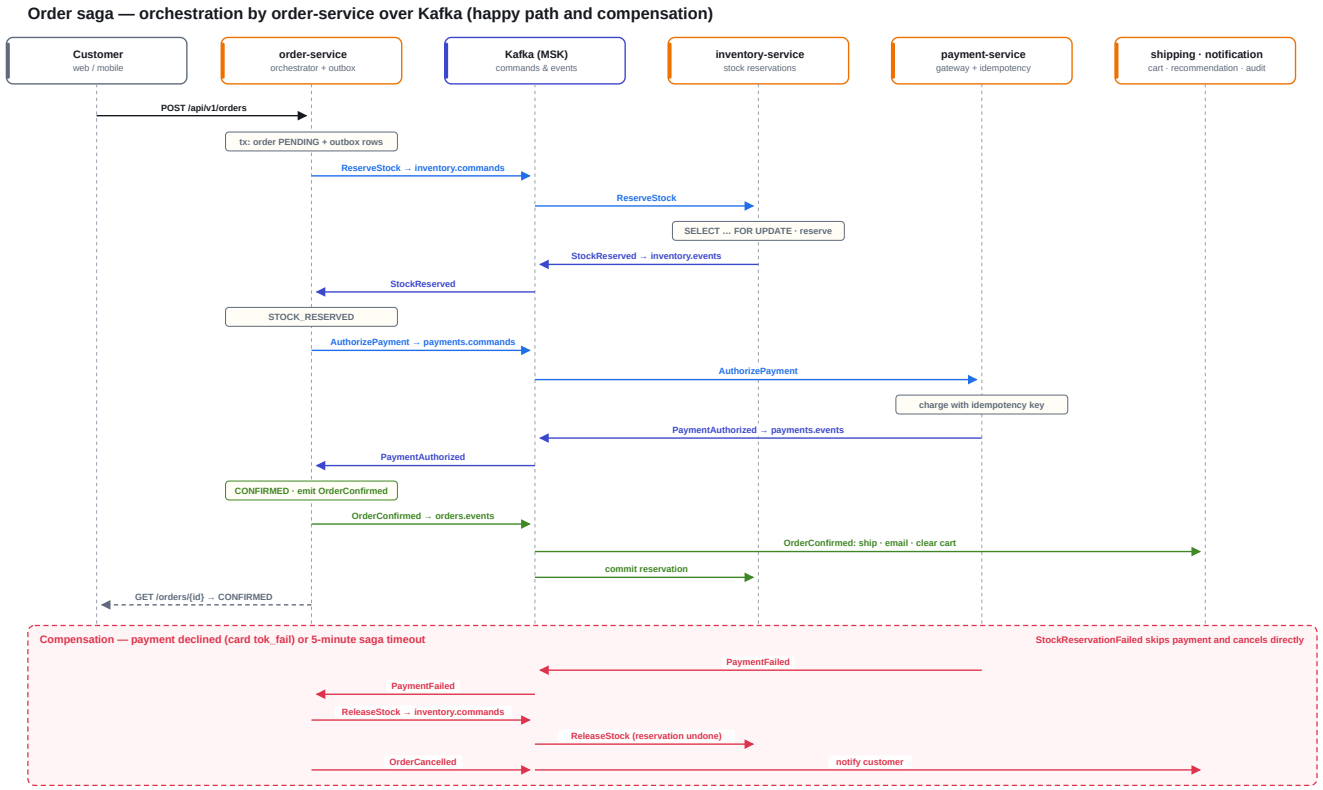
CI/CD, GitOps and Terraform



Jenkins builds and signs images. Git is the only path into a cluster: ArgoCD pulls the tag Jenkins committed. Promotion copies the same tag forward by pull request, so the artifact tested in staging is byte-for-byte the one in prod.

Two pipelines are kept separate on purpose. Jenkins owns application builds and never has rights to change infrastructure. Its EC2 instance role can only push to ECR and sign with the KMS key, so no access key exists to leak. GitHub Actions owns Terraform through two OIDC roles: a read-only role for plans on pull requests, and an apply role usable only from `main`. Prod apply also waits for a human approval in the GitHub `prod` environment.

Order saga



order-service is the single place that knows the state of every order. Each step is a Kafka command and reply, so any service can restart mid-checkout without losing the order. The red block is the compensation path the Postman suite exercises with the test card `tok_fail`.

Orchestration was chosen over choreography because the checkout flow has clear ordering and compensations. Keeping them in one state machine (`PENDING → STOCK_RESERVED → CONFIRMED | CANCELLED`) makes them testable and visible on one Grafana panel. A sweeper cancels any saga stuck for more than 5 minutes and runs its compensation.

Environments and cost

SETTING	DEV	STAGING	PROD
Domain	dev.ganga888.in	staging.ganga888.in	ganga888.in
AZs / NAT	2–3 AZs, 1 NAT	3 AZs, 1 NAT	3 AZs, 3 NAT
Nodes	2× t3.medium + Karpenter spot	2× t3.medium + Karpenter mix	system group + Karpenter on-demand/spot mix

SETTING	DEV	STAGING	PROD
Postgres	pod	RDS db.t4g.medium, single-AZ	RDS db.r6g.large, Multi-AZ, PITR
Redis	pod	cache.t4g.small + 1 replica	primary + 2 replicas, Multi-AZ
Kafka	Strimzi pod	MSK kafka.t3.small × 2	MSK kafka.m5.large × 3
Releases	auto on every build	promotion PR, canary	promotion PR + approval, canary
Runs	working hours only	on demand	always

> Rough monthly cost, inferred from public ap-south-1 list prices rather than a calculator run: dev about $80–150 if destroyed every night, staging about $400–600, prod about $1,800–3,000 before Savings Plans. MSK, RDS Multi-AZ and the three NAT gateways are the largest prod costs. The Jenkins EC2 host adds about $150 a month (t3.xlarge, 150 GB gp3 and its ALB), or roughly half that with `office_hours_schedule = true`. Check your own numbers in the AWS Pricing Calculator before the first prod apply.

Resilience and disaster recovery

- **Inside a zone:** liveness, readiness and startup probes. PodDisruptionBudgets keep at least 2 prod replicas during node drains. Circuit breakers (opossum) protect every outbound call.
- **Losing a zone:** pods spread across 3 AZs, RDS and Redis fail over automatically, and MSK replicates every partition 3 ways.
- **Losing the cluster:** recreate it with Terraform. ArgoCD redeploys from Git and Velero restores anything not in Git.
- **Losing data:** RDS point-in-time restore (RPO about 5 minutes), plus AWS Backup copies in a second region.
- **Targets for prod:** RTO 1 hour, RPO 15 minutes. The Ansible restore-drill playbook proves this every quarter, and `docs/runbooks/dr-restore.md` lists the steps.

Build order: from empty account to live site

1. **Bootstrap state.** `terraform/bootstrap` creates the S3 state bucket and lock table, once, with your admin credentials.

2. **Global resources.** `terraform/global` creates the Route 53 zone, ECR repos, GitHub OIDC roles, KMS keys, CloudTrail, GuardDuty and budgets. Then set your registrar's nameservers for ganga888.in to the four NS values it prints.
3. **Dev environment.** Open a PR on `terraform/envs/dev`, read the plan in the PR comment, and merge. GitHub Actions applies it.
4. **Bootstrap ArgoCD.** The morning workflow installs ArgoCD and applies `gitops/bootstrap/dev-root.yaml` (or run those two steps by hand). Everything else (ingress, monitoring, logging, the 15 services) syncs from Git.
5. **Start Jenkins on EC2.** Set `admin_cidrs` and apply `terraform/ci`, put the GitHub token in the `shopverse/ci/jenkins` secret, then run `ansible-playbook -i inventories/ci/hosts.yml playbooks/jenkins.yml`. Open `https://jenkins.ganga888.in`; the first build pushes all 15 images and bumps the dev tags.
6. **Verify.** Run the Newman suite against `api.dev.ganga888.in`, then k6 load while watching HPA and Karpenter in Grafana.
7. **Promote.** Apply staging and prod the same way, then promote image tags with the promote workflow. Prod applies and promotions wait for your approval.